
DIG. EVIDENCE. NO LIES.
// ANALYZE // EXTRACT // REVEAL
POSTMORTEM
> FORENSICS ANALYSIS TOOL <
CARVE FILES
EXTRACT DATA
MEMORY ANALYSIS
REPORT EVERYTHING
SCAN PROGRESS
100%
BINWALK [✓]
BULK_EXTRACTOR [✓]
STRINGS [✓]
VOLATILITY 3 [✓]
ARTIFACTS FOUND 2374
document.pdf
report.docx
credentials.json
chat_log.db
secret.txt
notes.md
wallet.dat
THE TRUTH IS IN THE DATA.
OFFSET 00 01 02 03 04 05 06 07
0001F4B0 3C 68 74 6D 6C 3E 0A 3C <html>
0001F4B8 68 65 61 64 3E 0A 3C 74 head>.<t
0001F4C0 69 74 6C 65 3E 52 65 70 itle>Rep
0001F4C8 6F 72 74 3C 2F 74 69 74 ort</tit
0001F4D0 6C 65 3E 0A 3C 2F 68 65 le>..</he
0001F4D8 61 64 3E 0A 3C 62 6F 64 ad>.<bod
0001F4E0 79 3E 0A 3C 70 3E 55 73 y>.<p>Us
0001F4E8 65 72 3A 20 61 64 6D 69 er: admi
0001F4F0 6E 0A 50 61 73 37 67 6F n.Passwo
0001F4F8 72 64 3A 20 68 75 6B 74 rd: hunt
0001F500 65 72 32 0A 46 69 6C 65 er2.File
1F508 3A 20 73 65 63 72 65 74 : secret
$ strings image.dd | grep -i "secret"
....
1056320: secret.txt
1056334: Top_Secret_Notes
1059872: password = hunter2
...
$ _
EVIDENCE LOCKED
STUDENT NAME Alon Agronov
STUDENT NO. s16
UNIT NX212
LECTURER Natalie Erez
ANALYZE.
EXTRACT.
REVEAL.

Overview:

This script uses various tools to automate forensic triage of a disk or memory image. It can:

- Carve files
- Extract readable data
- Run memory analysis
- Package all of the available information into folders, and log files that document all the actions taken.

Usage: sudo ./script.sh <image_file>

The scripts main:

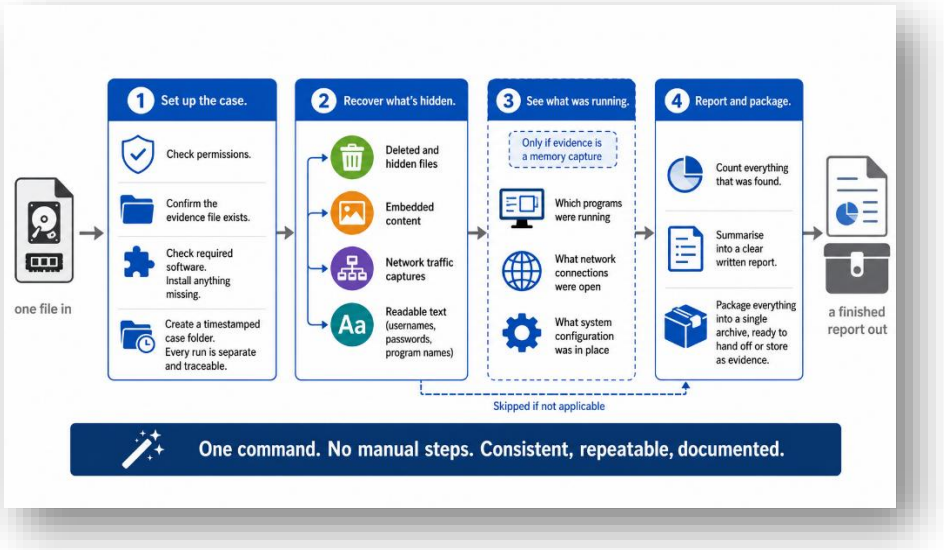
```
main() {                                                    # pretty straight forward
        START=$(date +%s)
        is_root
        select_file "$1"
        check_dependencies
        new_dir
        foremost_run
        binwalk_run
        bulk_run
        strings_run

        if detect_volatility && vol_check; then             # if Volatility is found - proceed to run plugins
            plugin_run windows.pslist pslist.txt
            plugin_run windows.netstat netstat.txt
            plugin_run windows.registry.hivelist hivelist.txt
        fi

        for dir in foremost binwalk bulk_extractor strings volatility; do
            count=$(file_count "$OUTDIR/$dir")
            echo "$dir: $count files"
        done

        END=$(date +%s)
        RUNTIME=$((END - START))
        echo "Analysis took $RUNTIME seconds"
        generate_report
        zip_it_ship_it
}

main "$@"
```



1. Prerequisites

1.1 User Privileges Check:

```
is_root(){                                                    # is user root?
    (( $EUID == 0 )) || die "[*] Please start the tool with root privileges"
}
```

First, since this script uses tools that demand elevated privileges, the script starts with checking whether the user has those privileges, or not

In a Linux machine, The EUID variable holds the numeric effective user ID. 0 means the user is root.

1.2 File selection:

```
select_file(){                                                # prompt the user to enter filename
    TARGET="$1"
    if [ -z "$TARGET" ]; then
        read -p "[*] Enter file to analyze: " TARGET
    fi
    [[ -f "$TARGET" ]] || die "[*] File does not exist: $TARGET"
}
```

Next, the user is supposed to enter a filename to analyze. For this, a small function I created checks if the user entered a filename while starting the script. If that is the case, the script starts running, if it's not, the user is advised to enter a filename for analysis. If the user enters an invalid filename, the script dies.

1.3 Helper functions

```
die() {                                                       # helper function to print errors
    echo "$1" >&2
    exit 1
}
```

This is more of a style preference, but it holds some practical uses. This function shows up when a function fails. Instead of echoing a message, sending it to stderr and exiting, I just use the function to save some lines. Also, it makes the code philosophy easier to understand (function runs cleanly – or "dies"). On top of that, it allows me to write quick "if" clauses with ||, as we will see later.

```
file_count() {
    find "$1" -type f 2>/dev/null | wc -l

    }
```

Used later, iterates each of the tools folders, and counts the number of files in them, to later document the numbers in the final report file.

```
zip_it_ship_it() {                                                    # zipping everything!
    local archive="${OUTDIR}.zip"
    echo "[*] Zipping the results..."
    zip -r "$archive" "$OUTDIR" > /dev/null 2>&1 && echo "[*] Archive created: $archive" || echo "[!] Zipping failed :( "

    }
```

Zips the entire analysis – using the command zip -r. $archive provides the name of the zip file, while $OUTDIR provides the baseline directory of the analysis.

1.4 tools & dependencies

```
declare -A tools=(
    [bulk_extractor]="bulk-extractor"
    [binwalk]="binwalk"
    [foremost]="foremost"
    [strings]="binutils"
)
```

An interesting small problem that appeared was that the conventional names of the tools were different from their names when using them in the console. For example, strings is a part of the “binutils” package, so checking if the **Package** is available on the machine, requires me to search for binutils, and not strings.

And so, I created a dictionary mapping that connects the name of the tool, and the name of its package. This way I can check dependencies on the package name and later use the same mapping to utilize the actual tool names.

```
tool_check() {
    command -v "$1" >/dev/null 2>&1
}
```

“command -v” is used to check the version of a tool inside a machine. I used it to check if each tool is available on the machine ->

```
check_dependencies() {                                                  # are any tools missing?
    local missing=()
    for tool in "${!tools[@]}"; do
        if ! tool_check "$tool"; then
            missing+=("${tools[$tool]}")
        fi
    done
    if (( ${#missing[@]} > 0 )); then
        echo "[*] Missing the tools: ${missing[*]} , proceeding to install them now..."
        apt-get update || die "[*] Failed to update package lists"
        apt-get install -y "${missing[@]}" || die "[*] Failed to install missing tools"
    fi
    # re-verifying
    for tool in "${!tools[@]}"; do
        tool_check "$tool" || die "[*] Failed to install: ${tools[$tool]}"
        done
}
```

This function Utilizes both the mapping, and the tool_check() function – it iterates each **Package** name, and checks if it is available. If it is NOT, it writes it into a "missing" list. If that list contains any tools, it then reflects that to the user and downloads those tools. For simple verification, the function then runs tool_check() again to make sure the download succeeded.

```
binwalk_run() {
    local bw_status

    printf '%s\n' "----------------------------------------"
    echo "[*] Running Binwalk..."

    binwalk "$TARGET" > "$OUTDIR/binwalk/signature_scan.txt" 2>&1


    binwalk -Me "$TARGET" -C "$OUTDIR/binwalk" \
        > "$OUTDIR/logs/binwalk.log" 2>&1
    bw_status=$?

    if (( bw_status != 0 )); then
        echo "[*] Binwalk completed with warnings or partial extraction issues."
        echo "[*] See: $OUTDIR/logs/binwalk.log"
    else
        echo "[*] Binwalk extracted files into: $OUTDIR/binwalk"
    fi

    echo "[*] Completed Binwalk."
    printf '%s\n' "----------------------------------------"
}
```

To execute binwalk, I passed $TARGET as the arg for it TWICE, once to get the signature scan (prints a table of what it found with offsets), and with the -Me flag, where M stands for "Matryoshka", which means binwalk scans files recursively, and -e stands for entropy, which checks and detects encrypted or compressed sections in binaries.

bw_status=$? Holds the exit code of the most recently completed command. In this case – binwalk -Me. This is used to check if the command passed perfectly, or rather had some problems. The if clause after takes care of the two cases.

```
bulk_run() {
    local be_status

    printf '%s\n' "----------------------------------------"
    echo "[*] Running Bulk Extractor..."

    bulk_extractor -o "$OUTDIR/bulk_extractor" "$TARGET" > "$OUTDIR/logs/bulk_extractor.log" 2>&1

    be_status=$?

    if (( be_status != 0 )); then
        echo "[*] Bulk Extractor completed with warnings - see /logs/bulk_extractor.log"
    fi

    local pcap="$OUTDIR/bulk_extractor/packets.pcap"
    if [[ -f "$pcap" ]]; then
        local size
        size=$(du -h "$pcap" | cut -f1)
        echo "[*] Network capture ($size) carved and can be found at: $pcap"
    else
        echo "[*] No network traffic packets.pcap found."
    fi

    echo "[*] Completed Bulk Extractor."
    printf '%s\n' "----------------------------------------"

    }
```

This bulk_extractor function is similar to the last function we saw but adds to the unique abilities of the tool. Here, if a .pcap file is found within the scan, it will be noted to the user, with its size and location.

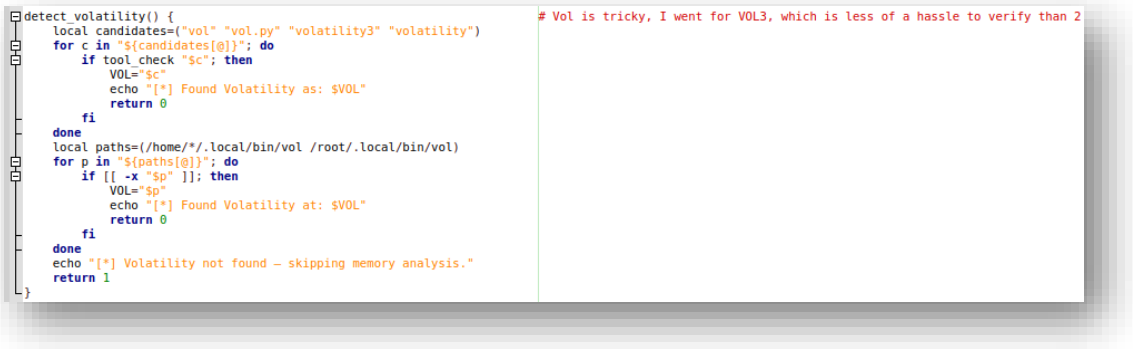
```
strings_run() {
    printf '%s\n' "----------------------------------------"
    echo "[*] Extracting readable strings..."
    local full="$OUTDIR/strings/unfiltered_strings.txt"

    strings "$TARGET" > "$full" 2>/dev/null

    local keywords=("password" "passwd" "username" "login" "token" "\.exe" "api" "secret" "mail" "confidential")
    for kw in "${keywords[@]}"; do
        grep -in "$kw" "$full" > "$OUTDIR/strings/${kw}.txt" 2>/dev/null
        local count
        count=$(wc -l < "$OUTDIR/strings/${kw}.txt")
        echo " '$kw' - $count hits"
    done
    echo "[*] Completed strings analysis."
    printf '%s\n' "----------------------------------------"
    }
```

To utilize strings to its full potential, I added some keywords that the command tries to find. Upon a hit, each keyword created its own .txt file with all the hits within the binaries of the file scanned. This information is conveyed to the user. Of course, an unfiltered strings scan is also saved, for the user to analyze.

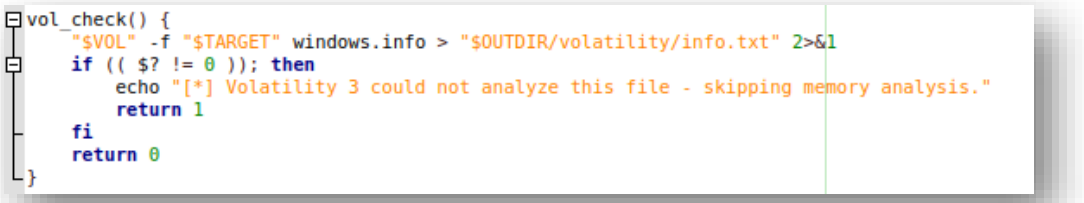
Volatility presented a few obstacles and made me choose between vol 2 and 3. In the end I chose to go with 3, which is more up to date, and can handle more recent OS' mem files.



Detecting Volatility is difficult because there's no single canonical install — different methods produce different command names and locations. My approach is a two-stage search. First, tool_check() probes a list of likely command names (vol, vol.py, volatility3, volatility); the first one found is stored in $VOL and used for every later call.

To double check, if nothing pops up there, I tried checking a default location for vol, which is located in .local/bin/vol. adding to that, I searched using a different mechanism ([[-x "$p"]]) which tests an absolute path directly.

! notice how I checked both in "home", and in "root". Running as root means $HOME is /root, and root's PATH does not include the invoking users /.local/bin. This way, an install under the "normal" account is invisible to the script.



This function prompts volatility to analyze the file given by the user (after verification that volatility is installed on the machine.). the function simply runs to extract windows.info and in an event the analysis fails, it's a soft fail that backs out of the volatility analysis.

```
plugin_run() {                                                          # running the plugins for VOL3...
    local plugin="$1" outfile="$2"
    echo "[*] Running $plugin..."
    "$VOL" -f "$TARGET" "$plugin" \
        > "$OUTDIR/volatility/$outfile" 2> "$OUTDIR/logs/vol_${outfile}.log" \
        || echo "[!] $plugin had issues"
}
```

Lastly, plugin_run is used to run different plugin commands in volatility 3, which is then saved to individual files.

This function runs ONLY if both vol_check() and detect_volatility() succeed, and it looks like this:

```
if detect_volatility && vol_check; then
    plugin_run windows.pslist pslist.txt
    plugin_run windows.netstat netstat.txt
    plugin_run windows.registry.hivelist hivelist.txt
fi
```

2. Baseline directory

```
new_dir() {
    local filename
    local base

    filename=$(basename -- "$TARGET")
    base="${filename%.*}"

    OUTDIR="results_${base}_$(date +%F_%H%M%S)"

    mkdir -p \
        "$OUTDIR/bulk_extractor" \
        "$OUTDIR/foremost" \
        "$OUTDIR/binwalk" \
        "$OUTDIR/strings" \
        "$OUTDIR/volatility" \
        "$OUTDIR/report" \
        "$OUTDIR/logs" \
        || die "[*] Failed to create output directories"
}
```

This function is used once to create the directory in which files will be written. Its name is a combination of the filename given, and the date the analysis took place.

It creates a sub directory for each tool, the logs, and a final report.

- *Please observe the use of the "die" function.*

3. Runtime & final report

3.1 Runtime

To capture the runtime accurately, I started the entire script with:

```
main() {
        START=$(date +%s)
```

+%s decides the format of the date – in this case, it transforms it into **epoch time** (seconds sins 1 January 1970 00:00 UTC).

At the end of the script, I added:

```
        END=$(date +%s)
        RUNTIME=$((END - START))
```

Which serves the same purpose as START. And finally I stated RUNTIME as END – START. This works since both of those variables are seconds.

3.2 Final Report

```
generate_report() {                                                    # final report
    local report="$OUTDIR/report/report.txt"

    cat > "$report" <<EOF
==========================================
      FORENSIC ANALYSIS REPORT
==========================================
Target file : $TARGET
Date        : $(date)
Runtime     : $RUNTIME seconds
Output dir  : $OUTDIR
==========================================

-- Files Extracted Per Tool --
EOF
    local total=0
    for dir in foremost binwalk bulk_extractor strings volatility; do
        local count
        count=$(file_count "$OUTDIR/$dir")
        printf "%-15s : %s files\n" "$dir" "$count" >> "$report"
        total=$((total + count))
    done
    echo "Total files : $total" >> "$report"

    echo "-- Strings Keyword Hits --" >> "$report"
    for f in "$OUTDIR/strings/"*.txt; do
        [[ "$f" == *unfiltered_strings.txt ]] && continue
        local name count
        name=$(basename "$f" .txt)
        count=$(wc -l < "$f")
        printf "%-15s : %s hits\n" "$name" "$count" >> "$report"
    done
    echo "[*] Report saved to: $report"
}
```

The final report contains information such as the target file, date, runtime, output directory, the number of files, and the strings keyword hits we extracted in strings_run().

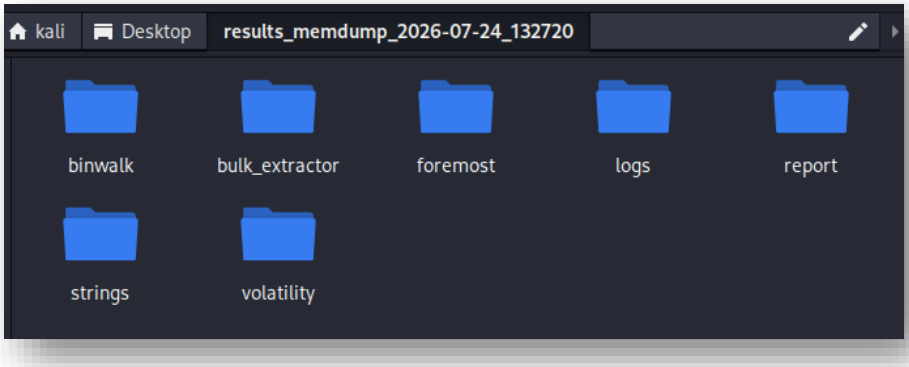
I used the cat > “” << EOF instead of echoing each line to save lines and be more effective.

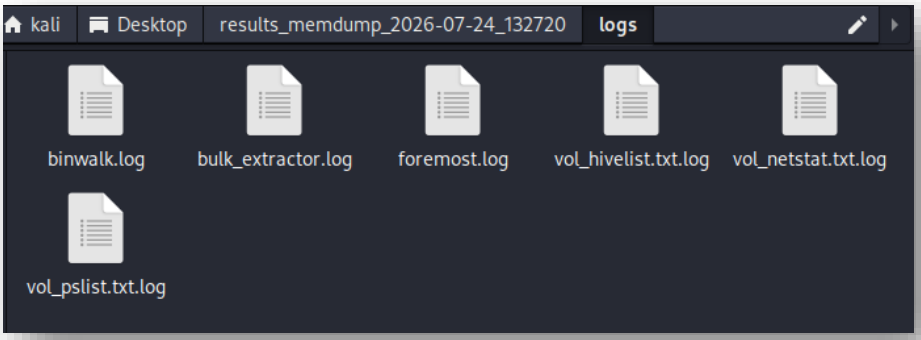
4. File analysis output

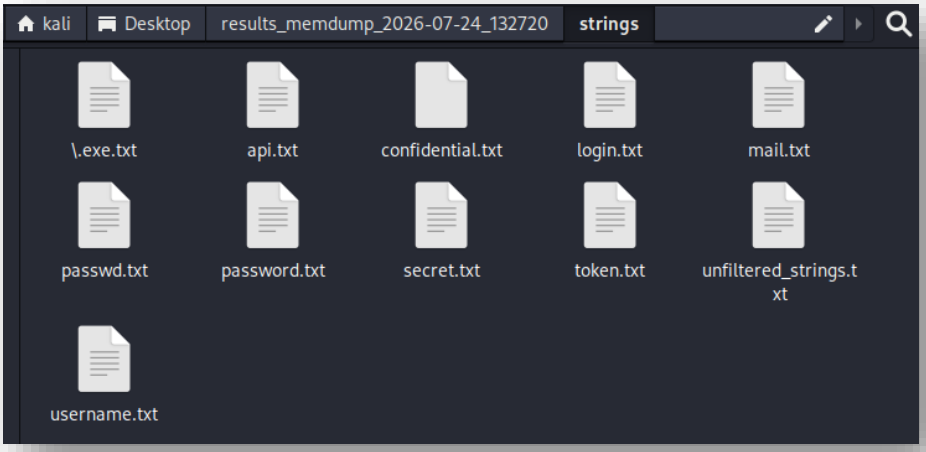
```
┌──(kali㉿kali)-[~/Desktop]
└─$ sudo ./Forensics.sh memdump.mem
----------------------------------------
[*] Running Foremost...
[*] Completed Foremost!
----------------------------------------
----------------------------------------
[*] Running Binwalk...
[*] Binwalk completed with warnings or partial extraction issues.
[*] See: results_memdump_2026-07-24_132720/logs/binwalk.log
[*] Completed Binwalk.
----------------------------------------
----------------------------------------
[*] Running Bulk Extractor...
[*] Network capture (104K) carved and can be found at: results_memdump_2026-07-24_132720/bulk_extractor/packets.pcap
[*] Completed Bulk Extractor.
----------------------------------------
----------------------------------------
```

```
[*] Extracting readable strings...
 'password' - 321 hits
 'passwd' - 11 hits
 'username' - 236 hits
 'login' - 139 hits
 'token' - 934 hits
 '\.exe' - 1013 hits
 'api' - 2915 hits
 'secret' - 75 hits
 'mail' - 231 hits
 'confidential' - 0 hits
[*] Completed strings analysis.
----------------------------------------
```

```
[*] Found Volatility at: /home/kali/.local/bin/vol
[*] Running windows.pslist...
[*] Running windows.netstat...
[!] windows.netstat had issues
[*] Running windows.registry.hivelist...
foremost: 182 files
binwalk: 1 files
bulk_extractor: 3185 files
strings: 11 files
volatility: 4 files
Analysis took 47 seconds
[*] Report saved to: results_memdump_2026-07-24_132720/report/report.txt
[*] Zipping the results...
[*] Archive created: results_memdump_2026-07-24_132720.zip
```


kali
Desktop
results_memdump_2026-07-24_132720
binwalk
bulk_extractor
foremost
logs
report
strings
volatility


kali
Desktop
results_memdump_2026-07-24_132720
logs
binwalk.log
bulk_extractor.log
foremost.log
vol_hivelist.txt.log
vol_netstat.txt.log
vol_pslist.txt.log


kali
Desktop
results_memdump_2026-07-24_132720
strings
\.exe.txt
api.txt
confidential.txt
login.txt
mail.txt
passwd.txt
password.txt
secret.txt
token.txt
unfiltered_strings.txt
username.txt

```
==========================================
      FORENSIC ANALYSIS REPORT
==========================================
Target file : memdump.mem
Date        : Fri Jul 24 01:28:07 PM EDT 2026
Runtime     : 47 seconds
Output dir  : results_memdump_2026-07-24_132720
==========================================

-- Files Extracted Per Tool --
foremost        : 182 files
binwalk         : 1 files
bulk_extractor  : 3185 files
strings         : 11 files
volatility      : 4 files
Total files : 3383
-- Strings Keyword Hits --
api             : 2915 hits
confidential    : 0 hits
\.exe           : 1013 hits
login           : 139 hits
mail            : 231 hits
passwd          : 11 hits
password        : 321 hits
secret          : 75 hits
token           : 934 hits
username        : 236 hits

```